
0. Platforma de dezvoltare Freescale Freedom KL46Z

Mediul de dezvoltare CodeWarrior v10.5

1. Obiectivul lucrării

Lucrarea își propune o introducere în studiul sistemelor cu procesor ARM, mai exact cu microprocesorul MKL46Z256VLL4 . Sunt descrise componentele platformei de dezvoltare Freescale Freedom KL46Z și rolul acestora.

Este descris mediul de dezvoltare CodeWarrior prin intermediul căruia se acționează asupra microprocesorului MKL46Z256VLL4, bazat pe procesorul ARM Cortex-M0. Sunt prezentate comenzile de bază pentru depanarea și simularea programelor în limbajul C, precum și crearea unui program, depanarea și rularea acestuia.

2. Breviar teoretic

2.1. Platforma de dezvoltare Freescale Freedom KL46Z

Platforma Freedom de la Freescale este o platformă de dezvoltare cu cost redus, compatibilă cu seria Kinetis de microcontrolere, care sunt bazate pe ARM Cortex-M0 + și nuclee Cortex-M4. Caracteristicile includ: acces facil la intrările și ieșirile MCU , operarea cu consum redus de energie, și o interfață încorporată de depanare numită OpenSDA de drag-and-drop, de programare flash, de comunicație serială și control înaintea depanării , toate printr-un simplu cablu USB. Este compatibilă cu accesorii Arduino™, permițând astfel numeroase posibilități de conectare de shield-uri care se portivesc cu platforma. Platforma Freedom este, de asemenea, susținută de o gamă largă de elemente de la Freescale, iar software-ul permite o inițializare mult mai rapidă, lăsând mai mult timp creării.

FRDM-KL46Z este o platformă de dezvoltare care conține microprocesorul Kinetis MKL46Z256VLL4 , construit pe baza procesorului ARM Cortex-M0. Dispozitivul MKL46Z256VLL4 are o frecvență maximă de funcționare de 48MHz, 256KB de flash, 32KB RAM, un controler USB cu viteză maximă, LCD și o mulțime de periferice analogice și digitale. Acest kit este un set de instrumente hardware și software de evaluare și dezvoltare. FRDM-KL46Z poate fi utilizat pentru a evalua: KL46, KL36, KL26 și dispozitivele din seria L: KL16 Kinetis. Hardware FRDM-KL46Z este compatibilă cu structura pinilor Arduino R3, oferind o gamă largă de opțiuni de extindere a platformei.

Platforma include un segment LCD de 4 cifre, un accelerometru digital de 3 axe, magnetometru, slider tactil capacitiv și senzor de lumină ambientală. Aceasta placă are un adaptor standard deschis, încorporat, serial și de depanare cunoscut sub numele de OpenSDA. Acest circuit oferă mai multe opțiuni pentru comunicații seriale, programare flash și control la rularea depanării.

Caracteristicile FRDM-KL46Z includ:

- Microprocesor MKL46Z256VLL4(frecvența maximă 48 MHz, 256KB flash, 32 KB RAM);
- Dublu rol de interfață USB cu conector USB mini-B;
- Open SDA;
- Modulul LCD 7 segmente de 4 cifre;
- Slider tactil capacitiv;
- Senzor de lumină ambientală;
- Accelerometru MMA8451Q;
- Magnetometru MMA3110;
- 2 LED-uri, roșu și verde;
- 2 butoane;
- Opțiuni flexibile de alimentare - USB, baterie rotundă, sursă externă;
- Baterie pregătită, puncte de acces pentru măsurarea puterii;
- Acces ușor la I / O MCU-ului prin intermediul conectorilor Arduino TM R3 compatibili cu I / O;
- Interfață programabilă OpenSDA de depanare cu mai multe aplicații disponibile, inclusiv:
 - Dispozitiv flash de stocare în masă a interfeței programabile;
 - Interfață P & E de depanare oferă controlul la rularea depanării și compatibilitate cu instrumente IDE;
 - Interfață CMSIS-DAP: noul standard ARM pentru interfață de depanare integrată;
 - Aplicație de înregistrare a datelor.
- Compatibilitate cu Arduino R3.

Figura 1 prezintă o diagramă bloc a designului FRDM-KL46Z. Componentele primare și plasarea lor pe ansamblul hardware sunt evidențiate în figura 2.

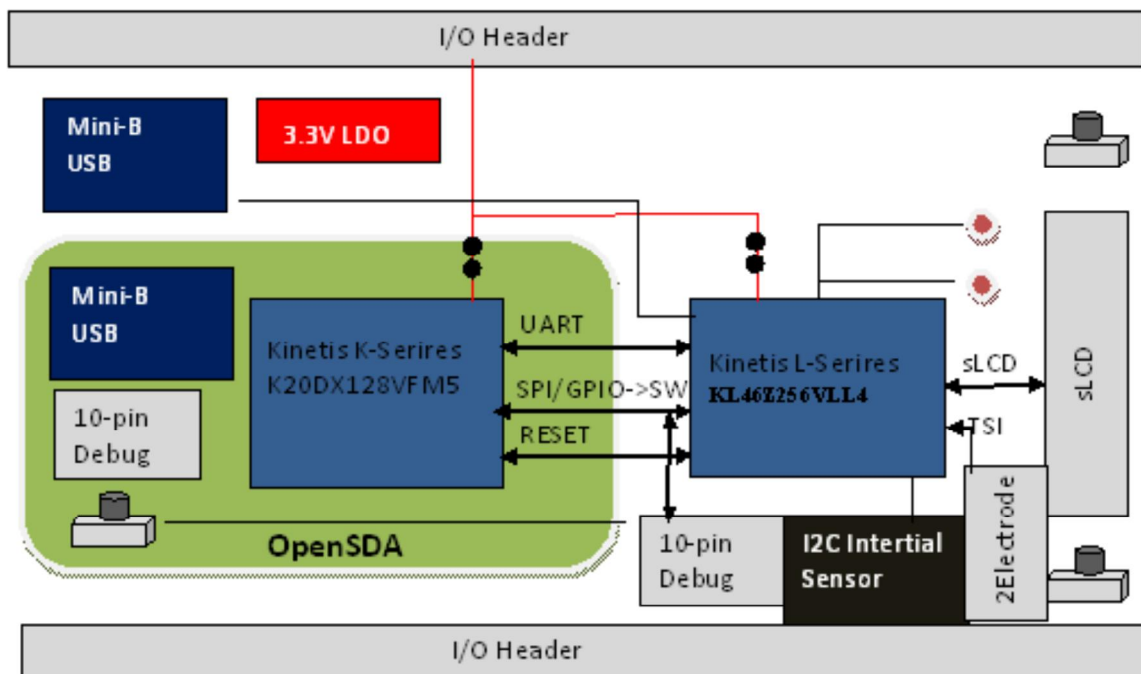


Figura 1. Diagrama bloc FRDM KL46Z

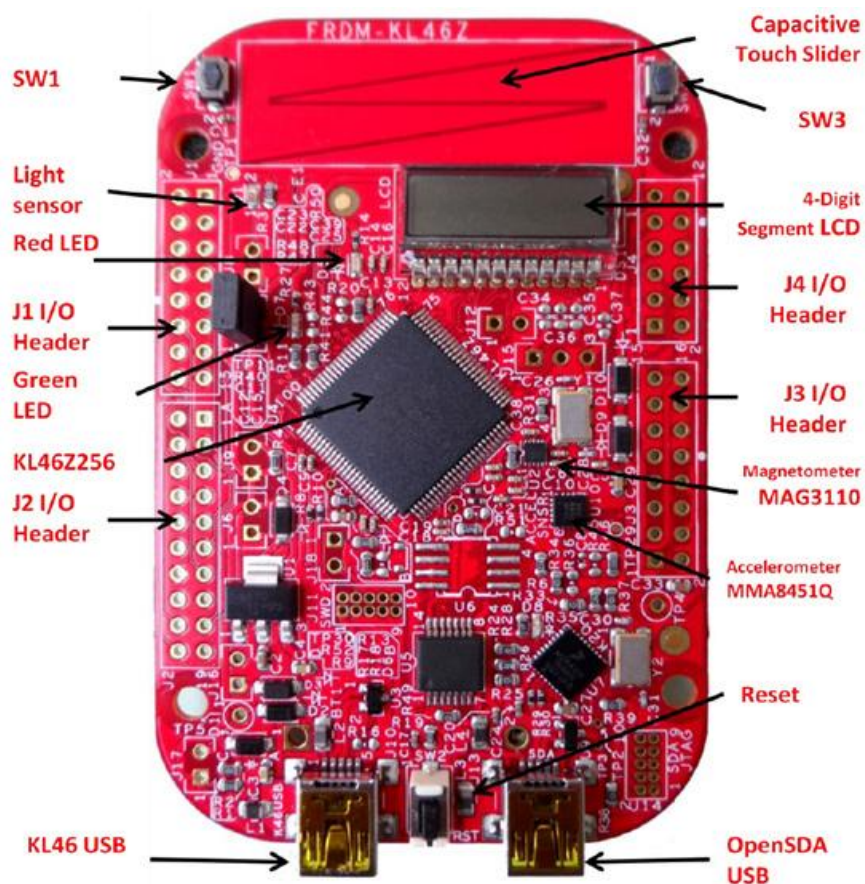


Figura 2. Plasarea componentelor principale pe platforma FRDM KL46Z

2.1.1. Alimentarea platformei KL46Z

Există mai multe opțiuni de alimentare a platformei FRDM-KL46Z. Aceasta poate fi alimentată de la oricare dintre cei doi conectori USB, pinul Vin de pe baza I/O, de la o baterie rotundă de pe platformă sau de la o sursă de 1.71V-3.6V la pinul de 3.3V de la baza I/O. Alimentările de la USB și Vin sunt reglate pe platformă cu ajutorul unui regulator liniar de 3.3V pentru a produce sursa principală de alimentare. Celelalte două surse nu sunt reglate pe platformă. Tabelul 1 oferă detaliile operaționale și cerințele pentru sursele de alimentare.

Sursa de alimentare	Intervalul de valori	OpenSDA operațional?	Reglate pe platformă?
OpenSDA USB	5V	Da	Da
K20 USB	5V	Nu	Da
Pinul Vin	4.3-9V	Nu	Da
Pinul 3.3V	1.71V-3.6V	Nu	Nu
Baterie rotundă	1.71V-3.6V	Nu	Nu

Tabelul 1: Alimentarea platformei KL46Z

De reținut că circuitul OpenSDA este operațional doar atunci când un cablu USB este conectat și alimentat cu energie pe USB-ul OpenSDA. Cu toate acestea, circuitul de protecție este așezat astfel încât să permită ca mai multe surse să fie alimentate o dată.

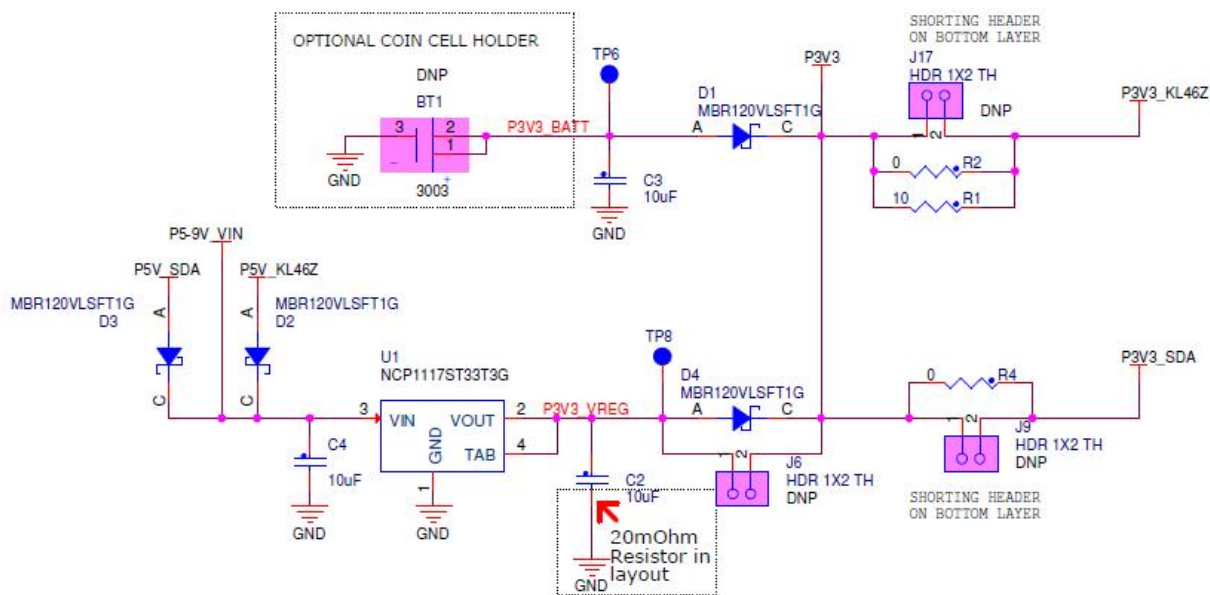


Figura 3 Schematicul sursei de alimentare

2.1.2. Adaptorul Serial și de depanare (OpenSDA)

OpenSDA este un adaptor serial și de depanare cu standard deschis. Este o punte de comunicare serială și de depanare între o gazdă USB și un procesor încorporat ținută, așa cum se arată în figura 4. Circuitul hardware se bazează pe un microcontroler de familie Freescale Kinetis K20 (MCU) încorporat, cu 128 KB flash și un controler USB integrat. OpenSDA dispune de un dispozitiv de stocare în masă (MSD) bootloader, care prevede un mecanism rapid și ușor de încărcare a diverselor aplicații OpenSDA, cum ar fi programatori flash, interfețe de depanare și de control la rulare, convertoare seriale la USB, etc.

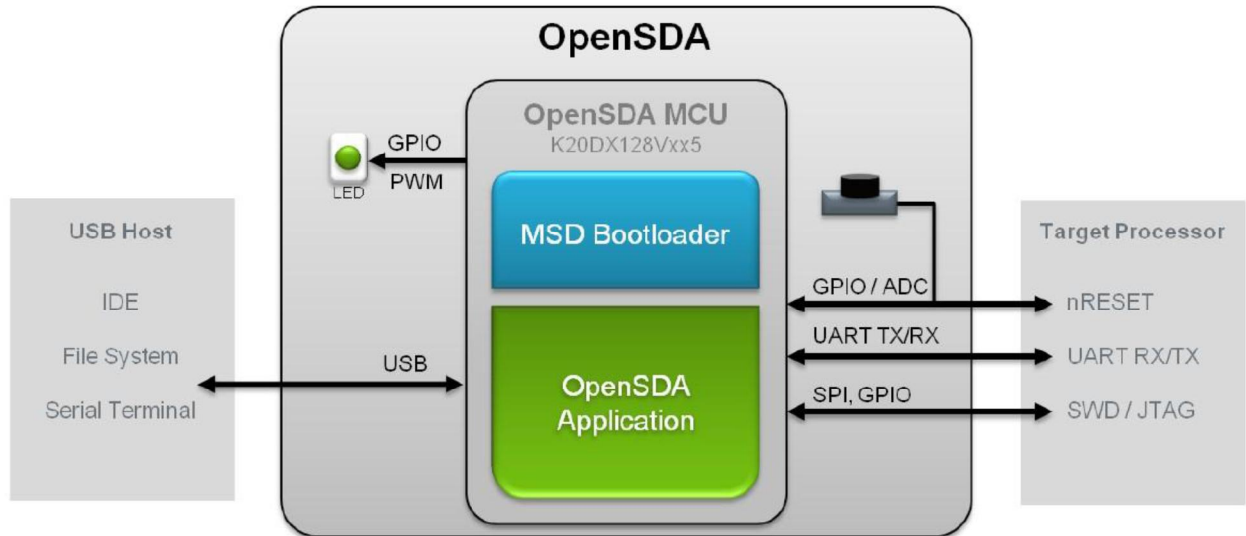


Figura 4 Diagrama bloc a adaptorului OpenSDA

OpenSDA este gestionat de un microcontroler Kinetis K20 construit pe ARM® Cortex™-M4 core. Circuitul OpenSDA include un LED de stare (D8) și un buton (SW2). Butonul activează semnalul de resetare la MCU KL46. Acesta poate fi de asemenea folosit pentru a plasa circuitul OpenSDA în modul Bootloader. Semnalele SPI și GPIO oferă o interfață la oricare port de depanare SWD al K20. În plus, conexiunile de semnal sunt disponibile pentru a implementa un canal UART serial. Circuitul OpenSDA primește energie atunci când conectorul J13 USB este conectat la un host USB.

2.1.3. Interfața de depanare

Semnale cu capacitate SPI și GPIO sunt utilizate pentru a se conecta direct la SWD al KL46. Aceste semnale sunt aduse, de asemenea, la un standard de 10-pini (0.05 ") Conector de depanare Cortex (J11). Este posibil să se izoleze microcontrolerul KL46 de circuitul OpenSDA prin utilizarea J11 și să se conecteze la un microcontroler din exterior. Pentru a realiza acest lucru, se taie legătura din partea de jos a PCB care conectează pinul 2 J18, la pinul 2 J11. Acest lucru va deconecta pinul SWD_CLK la KL46, astfel că nu va interfera cu comunicațiile de la un microcontroler exterior conectat la J11.

SWD CONNECTOR

SHORTING HEADER ON BOTTOM LAYER

Jumper is shorted by a cut-trace on bottom layer. Cutting the trace will effectively isolate the on-board MCU from the OpenSDA debug interface.

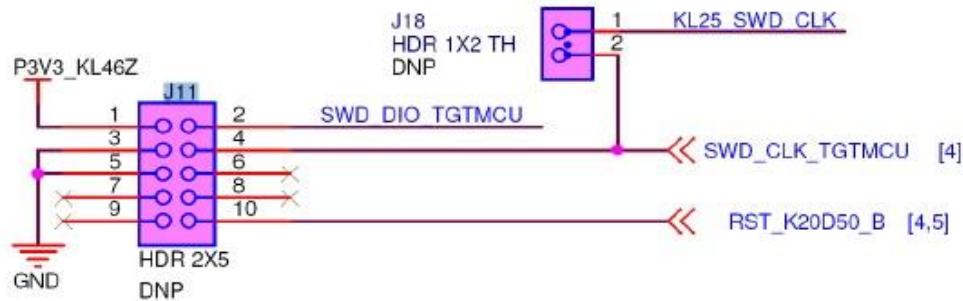


Figura 5 Conectorul de depanare SWD

De reținut că J11 nu este nepopulat în mod implicit. Un Samtec FTSH-105-02-FD sau conector compatibil poate fi adăugat la J11 prin găuri de conectare. Un cablu de împerechere, cum ar fi un cablu Samtec FFSD IDC, poate fi apoi utilizat pentru a conecta la OpenSDA de la FRDM-KL46Z la un conector SWD din exterior.

2.1.4. Portul virtual de serializare

O conexiune port serială este disponibilă între OpenSDA MCU și pinii PTA1 și PTA2 ai KL46. Mai multe dintre aplicațiile OpenSDA implicite furnizate de Freescale, inclusiv MSD Flash Programator și Debug P & E, oferă o interfață USB a clasei de dispozitive de comunicații (CDC), care face legătura de comunicare serială între USB și această interfață serială pe K20.

2.1.5. Microcontrolerul MKL46Z4

Microcontrolerul țintă al FRDM-KL46Z este MKL46Z256VLL4, un dispozitiv Kinetis seria L într-un pachet LQFP 100. Caracteristicile KL46Z MCU includ:

- 32-bit ARM Cortex-M0 + nuclee:
 - Funcționare de până la 48 MHz;
 - Un singur ciclu de acces rapid la portul I/O;
- Memorii;
- 256KB flash;
- 32 KB SRAM;
- Sistemul integrează:

- Controlere de gestionare a energiei și de mod;
- Motor de manipulare pe bit pentru citire-modificare-scriere operațiuni periferice;
- Controler de acces direct la memorie (DMA);
- Funcționare în mod corespunzător cu calculatorul (COP) timer Watchdog;
- Surgeri reduse ale unității de reactivare.
- Clock-uri:
 - Modulul de generare de ceas cu FLL și PLL pentru sistem și generarea de ceas CPU;
 - Ceas de referință intern de 4MHz și 32KHz;
 - Oscilatorul sistemului care suportă cristal extern sau rezonator;
 - Oscilator de joasă putere de 1kHz RC pentru RTC și COP watchdog;
- Periferice analogice:
 - ADC pe 16 biți cu aproximații succesive și cu suport DMA;
 - DAC pe 12 biți cu suport DMA;
 - Comparator de mare viteză.
- Periferice de comunicație:
 - Un interchip integrat de sunet (I2S) interfata audio (SAI);
 - Două interfețe periferice seriale de 8 biți (SPI);
 - Controler USB cu dublu rol cu transceiver integrat FS/LS;
 - Regulator de tensiune USB;
 - Două module I2C;
 - Un UART de joasă putere și două module standard UART.
- Timere:
 - Un modul Timer/PWM 6 canale;
 - Două module Timer/PWM 2 canale;
 - Timer periodic de întrerupere 2 canale (PIT);
 - Ceas în timp real (RTC);
 - Timer de joasă putere (LPT);
 - Timer tick sistem.
- Interfața om-mașină:
 - Controler LCD 7-segmente. Segmentul maxim este 8x47 sau 4x51;
 - Controler de intrare / ieșire de uz general;
 - Slider tactil capaciativ.

2.1.6. Sursa clock-ului

Microcontrolere Kinetis KL46 dispun de un oscilator on-chip compatibil cu trei game de frecvențe de intrare de cristal sau rezonator: 32-40 kHz (modul frecvențelor scăzute), 3-8 MHz (modul frecvențelor mari, gama scăzută.) și 8-32 MHz (modul frecvențelor ridicate, gama mare). KL46Z256 pe FRDM-KL46Z este cronometrat de la un cristal de 8 MHz .

2.1.7. Interfața USB

Microcontrolerele Kinetis KL46 dispun de un controler USB cu dublu rol cu transceivere de mare viteză și de viteză redusă pe cip. Interfața USB de pe FRDM-KL46Z este configurată ca un dispozitiv USB de mare viteză.

VREGIN trebuie alimentat pentru a activa circuitul intern USB (prin punte J7).

2.1.8. Portul serial

Semnalele primare de interfață port serial sunt PTA1 UART0 RX și PTA2 UART0_TX. Aceste semnale sunt conectate OpenSDA.

2.1.9. Reset

Semnalul RESET este conectat extern la un buton, SW2, și de asemenea la circuitul OpenSDA. Butonul de resetare poate fi folosit pentru a forța un eveniment de resetare externă în microcontrolerul țintă. Butonul de resetare poate fi de asemenea utilizat pentru a forța circuitul OpenSDA în modul bootloader.

2.1.10. Depanare

Interfața de depanare unică pe toate dispozitivele Kinetis seria L este un port serial pe fire de depanare (SWD). Controlerul principal al acestei interfațe pe FRDM-KL46Z este circuitul OpenSDA aflat pe platformă. Cu toate acestea, un conector cortex nepopulat de depanare cu 10-pini (0.05 "), J11, oferă acces la semnalele SWD. Samtec FTSH-105-02-FD sau conectorii compatibili pot fi adăugați la J11 prin găuri de conector de depanare pentru a permite unui cablu de depanare extern să fie conectat.

2.1.11. Segmentul LCD

FRDM-KL46Z utilizează un afișaj de 4 cifre (LUMEX LCD-S401M16KR) de 4x8 segmente. Următorul tabel arată conexiunea de la KL46 la ecran S401.

Pini S041	Pini LCD KL46Z
1	LCD_P40 (COM0)
2	LCD_P52 (COM1)
3	LCD_P19 (COM2)

4	LCD_P18 (COM3)
5	LCD_P37
6	LCD_P17
7	LCD_P7
8	LCD_P8
9	LCD_P53
10	LCD_P38
11	LCD_P10
12	LCD_P11

Tabelul 2 Conexiunile segmentului LCD



Figura 6 Aspectul segmentelor S041

2.1.12. Silder-ul tactil capacitiv

Două semnale de intrare de detectare a atingerii (STI), TSI0_CH9/PTB16, și TSI0_CH10/PTB17 sunt conectate la electrozi capacitivi configurați ca un slider tactil. Software-ul Touch Sense de la Freescale (TSS), oferă o bibliotecă software pentru implementarea senzorului tactil capacitiv.

2.1.13. Accelerometru cu trei axe

Un accelerometru de la Freescale, MMA8451Q de mică putere, cu trei axe este conectat printr-un bus I2C și două semnale GPIO așa cum se arată în tabelul 3 de mai jos. În mod implicit, adresa I2C este 0x1D.

MMA8451Q	KL46
SCL	PTE25/TPM0_CH1/I2C0_SDA
SDA	PTE24/TPM0_CH0/I2C0_SCL
INT1_ACCEL	PTC5/LLWU_P9
INT2_ACCEL	PTD1 (comun cu INT2_MAG)

Tabelul 3 Conectarea semnalelor accelerometrului

2.1.14. Magnetometru digital cu trei axe

Un magnetometru digital de la Freescale, MAG3110, cu trei axe, este conectat printr-un bus I2C și unul din semnalele GPIO așa cum se arată în tabelul 4 de mai jos.

MAG3110	KL46
SCL	PTE25/TPM0_CH1/I2C0_SDA
SDA	PTE24/TPM0_CH0/I2C0_SCL
INT1_MAG	PTD1 (comun cu INT2_ACCEL) poate fi izolat prin îndepărtarea R50

Tabelul 4 Conectarea semnalelor magnetometrului

2.1.15. LED-uri

Două LED-uri, LED-ul verde este PWM capabil, conexiunile semnalelor sunt prezentate în tabelul 5.

LED	KL46
Green	PTD5
Red	PTE29/TPM0_CH2

Tabelul 5 Conexiunile semnalelor LED-urilor

2.1.16. Senzor de lumină vizibilă

FRDM-KL46Z are un senzor de lumină vizibilă, care este conectat la ADC0_SE3.

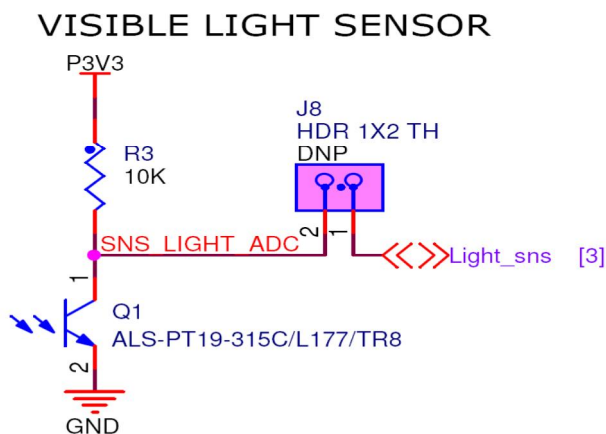


Figura 7 Schematicul senzorului de lumină

2.1.17. Conectori de intrare / ieșire

Microcontrolerul MKL46Z256VLL4 este ambalat într-un pachet LQFP de 100-pini. Unii pini sunt utilizați în circuitele de la platformă, dar multe sunt conectate direct la una din cele patru header-uri I / O.

Pinii de pe microcontroler-ul KL46 sunt numiți după funcția pinului portului de intrare / ieșire. De exemplu, pinul 1 de pe portul A este menționat ca PTA1. Numele pinului de conectare, de intrare/ieșire îi este dat același nume ca și cel al pinului conectat la KL46Z, acolo unde este cazul.

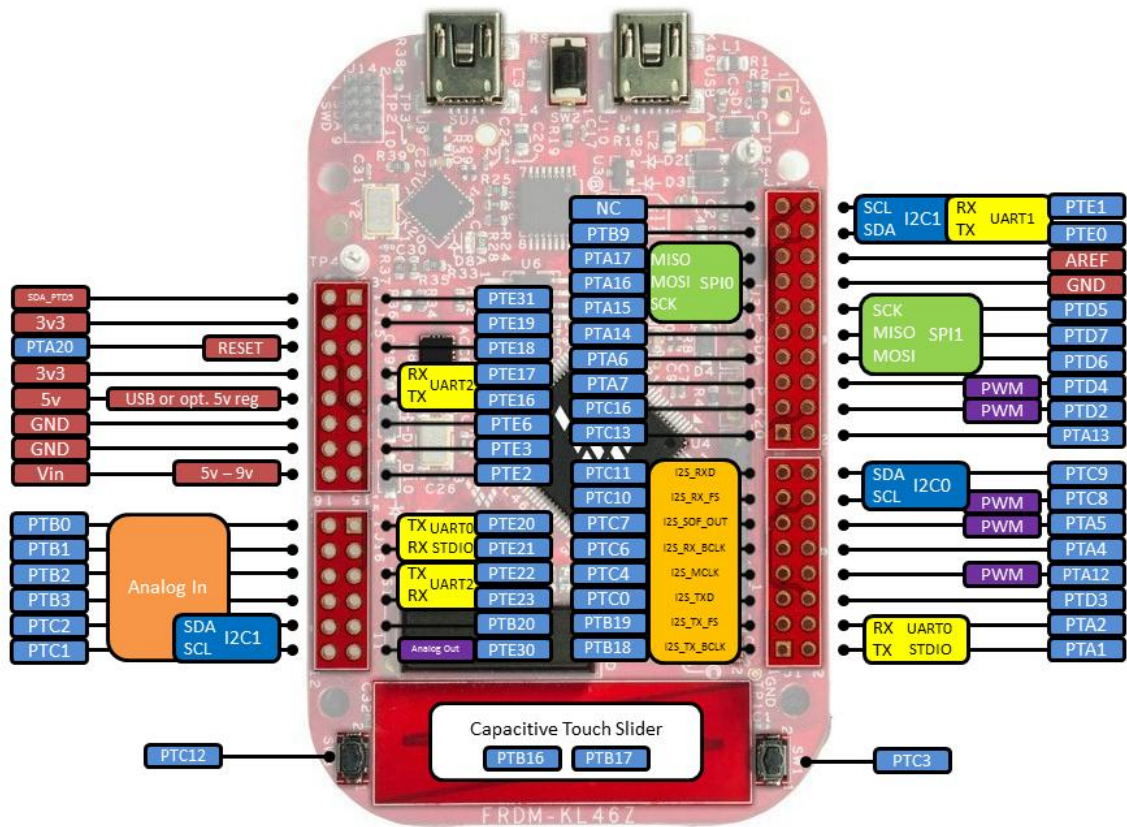


Figura 8 Pinii FRDM-KL46Z

2.1.18. Compatibilitatea cu Arduino

Headerele I / O pe FRDM-KL46Z sunt aranjate pentru a permite compatibilitatea cu plăci periferice (cunoscută sub numele de shield-uri), care se conectează la Arduino™ și plăci compatibile cu microcontroler Arduino. Rândurile exterioare de pini (dublurile chiar numerotate) pe header împărtășesc aceeași distanță mecanică și plasare ca headerele I / O pe standardul Arduino Revizia 3 (R3).

2.2. Mediul de dezvoltare CodeWarrior v10.5

2.2.1. Prezentarea mediului de dezvoltare

CodeWarrior este un mediu de dezvoltare integrat (IDE) pentru crearea de software care ruleaza pe o serie de sisteme embedded. Înainte de achiziționarea produsului de către Freescale Semiconductor, au existat versiuni pentru Macintosh, Microsoft Windows, Linux, Solaris, PlayStation 2, Nintendo GameCube, Nintendo DS, Wii, Palm OS, sistemul de operare Symbian, și chiar pentru BeOS.

În prezent, C, C ++, și limbajul de asamblare sunt în centrul instrumentelor mediului de dezvoltare, deși înainte Metrowerks a fost achiziționat de către Freescale, iar acesta continea versiuni de CodeWarrior care includeau ca limbaje de programare Pascal, Object Pascal, Objective-C, și compilatoare Java, de asemenea.

CodeWarrior a fost inițial dezvoltat de Metrowerks și era bazat pe un compilator C și avea ca mediu Motorola 68K, dezvoltat de Andreas Hommel și licențiat la Metrowerks. Primele versiuni ale CodeWarrior au vizat PowerPC Macintosh, cu o mare parte din dezvoltare realizată de un grup din echipa originală THINK C. La fel ca THINK C, care a fost cunoscut pentru timpul scurt de compliere, CodeWarrior a fost mai rapid decât Macintosh Programmer de la Workshop (MPW), instrument de dezvoltare scris de Apple.

Freescale CodeWarrior Development Studio pentru microcontrolere este un mediu de dezvoltare integrat IDE Eclipse de la Freescale care, printre altele, sprijină familia Freescale Kinetis MCU (Cortex-M4).

CodeWarrior IDE are o interfață de utilizator practic identică în mai multe gazde. Din acest motiv, ilustrarea elementelor comune de interfață folosesc imagini de la orice gazdă. Cu toate acestea, unele elemente de interfață sunt unice pentru o anumită gazdă. În astfel de cazuri, în mod clar imaginile etichetate identifica gazda specifică.

CodeWarrior IDE oferă un software de dezvoltare cu o suită de instrumente eficiente și flexibile. Acest subiect explică ciclul de dezvoltare software și avantajele utilizării IDE CodeWarrior pentru dezvoltare.

Un programator urmează un proces general pentru a dezvolta un proiect:

1. Se începe cu o idee pentru un nou software
2. Se implementează noua idee în cod sursă
3. Se compilează codul sursă în cod mașină
4. Se leagă codul mașină și se creează un fișier executabil
5. Se corectează erorile(debug)
6. Se compilează, se leagă și se lansează fișierul executabil final.

2.2.2. Avantajele CodeWarrior

- Se poate dezvolta pe mai multe platforme

Dezvolta software-ul pentru a rula pe sisteme de operare multiple, sau pentru a folosi mai multe gazde pentru dezvoltarea aceluiaș proiect software. CodeWarrior IDE rulează pe sisteme de operare populare, cum ar fi Windows, Solaris, și Linux. Acesta utilizează practic aceeași interfață grafică cu utilizatorul (GUI) pentru toate produsele bazate pe Freescale Eclipse.

- Suport pentru mai multe limbaje de programare

Se alege din mai multe limbaje de programare când se dezvoltă un software. CodeWarrior IDE suportă limbaje de nivel înalt, cum ar fi C, C++ și Java, precum și limbaj de asamblare pentru majoritatea procesoarelor.

- Mediu de dezvoltare în concordanță

Software-ul de port la noi procesoare, fără a fi nevoie să înveți noile instrumente sau să pierzi un cod de bază existent. CodeWarrior IDE suportă multe desktop-uri comune și familii de procesoare încorporate, cum ar fi x86, PowerPC, și MIPS.

- Instrument suport de conectare

Extinde capacitățile de CodeWarrior IDE prin adăugarea unui instrument de plug-in care sprijină noi caracteristici. CodeWarrior IDE acceptă în prezent plug-in-uri pentru compilatoare, elemente de legatură, pre-elemente de legătură, post-elemente de legătură, panouri preferențiale, și alte instrumente. Plug-in-urile fac posibil ca IDE CodeWarrior să proceseze diferite limbi și să suporte diferite familii de procesoare.

2.2.3. Prezentarea modului de lucru în CodeWarrior v10.5

Pentru crearea unui nou proiect în cadrul programului CodeWarrior, pentru compilarea acestuia și încărcarea acestui program pe placa de dezvoltare de la Freescale, FRDM KL46Z, este necesară parcurgerea următoarelor etape:

1. Se deschide programul CodeWarrior aflat pe desktop, iar din fereastra principală se selectează: **File -> New -> Bareboard Project**

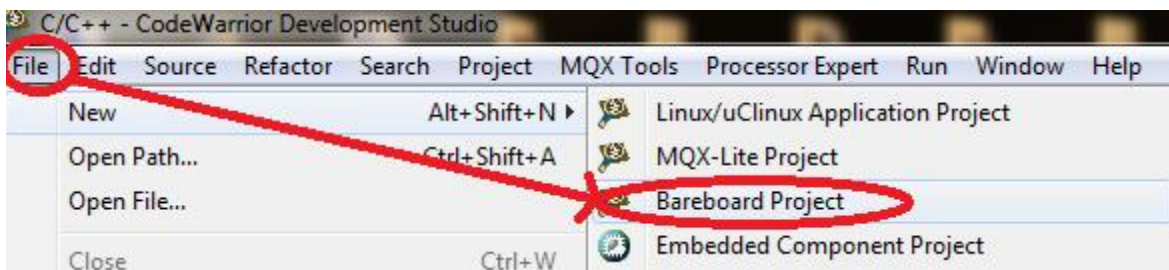


Figura 9. Modul de creare al unui nou proiect

2. Se va deschide o fereastră în care trebuie denumit proiectul și aleasă calea unde se va salva noul program. Apoi se apasă butonul **Next**.

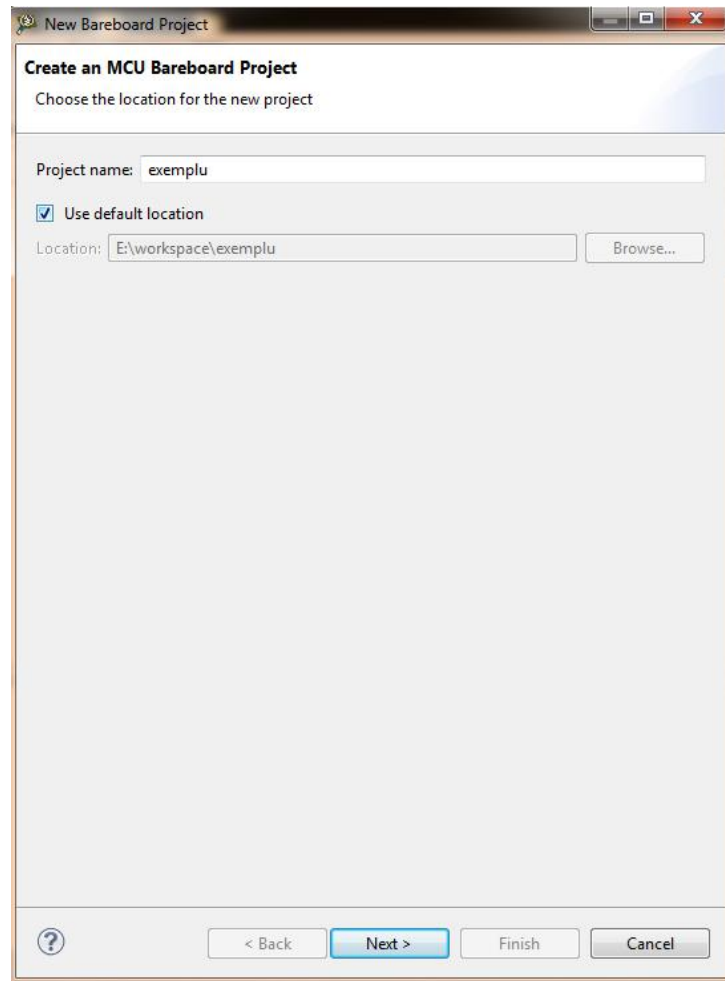


Figura 10. Fereastra de dialog în care se selectează numele și locația noului proiect

3. În continuare, în fereastra nou deschisă, se selectează tipul procesorului folosit în aplicație. În cazul nostru se folosește procesorul **MKL46Z256**, produs de Freescale, din seria Kinetis, familia KL46Z.

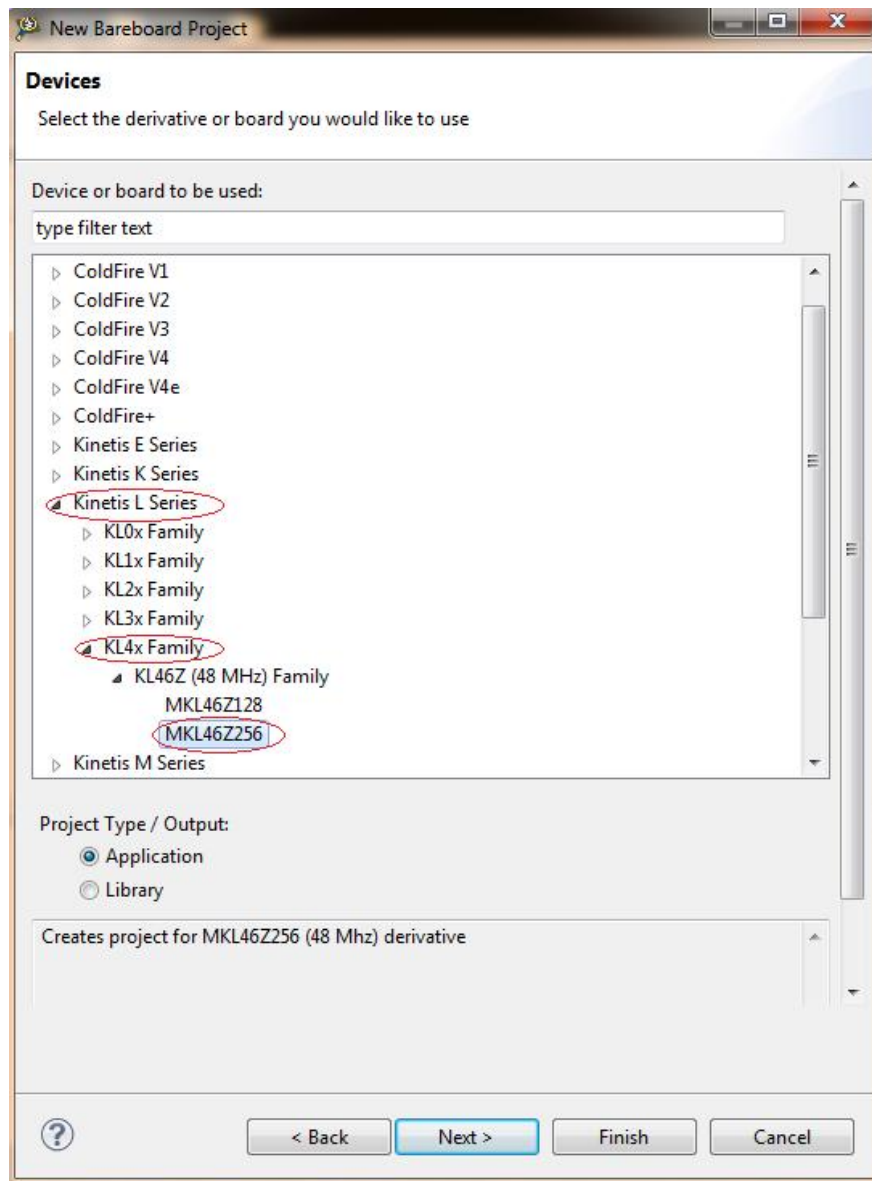


Figura 11. Selectarea procesorului folosit în aplicație

4. În fereastra nou deschisă se alege tipul conexiunii folosite, în cazul nostru **OpenSDA**, după care se apasă butonul **Next**.

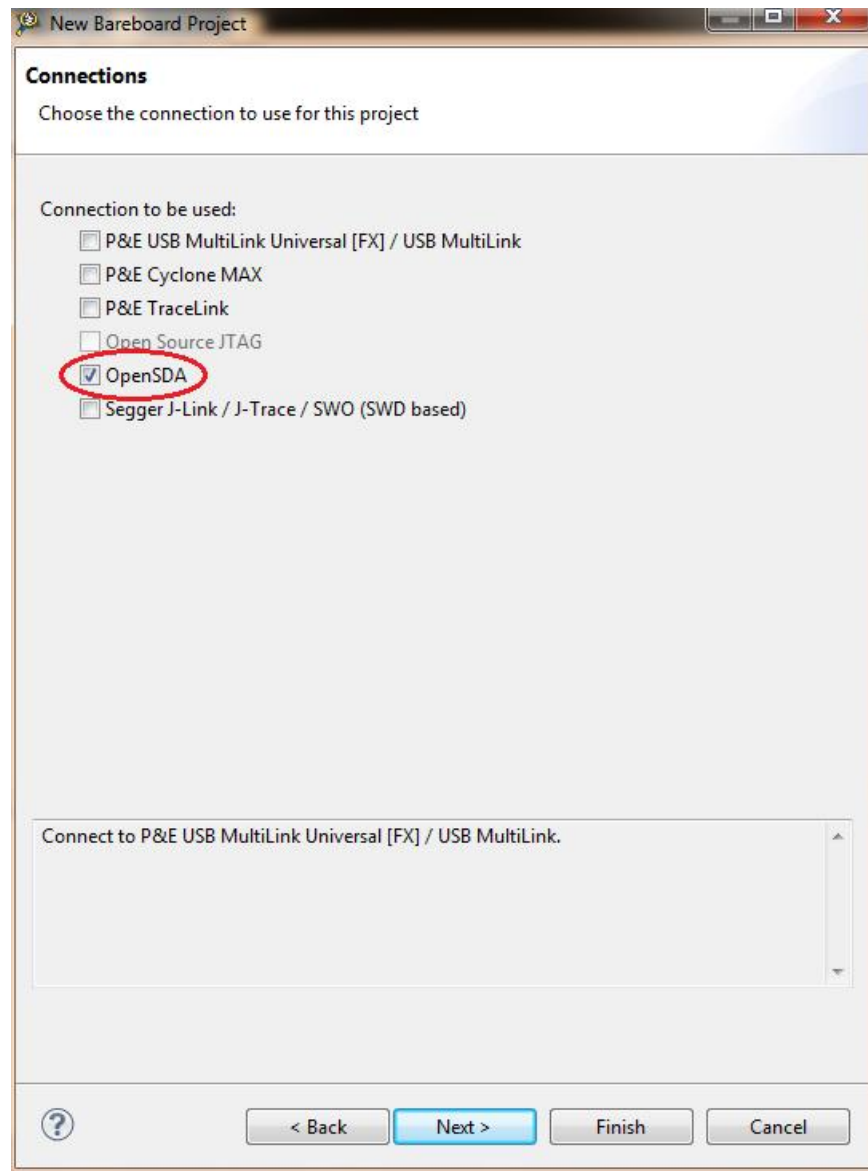


Figura 12. Selectarea conexiunii

5. Următorul pas îl constituie alegerea limbajului în care se dorește scrierea programului, suportul I/O. În aplicația noastră se va folosi limbajul **C** și **UART** ca suport pentru I/O.

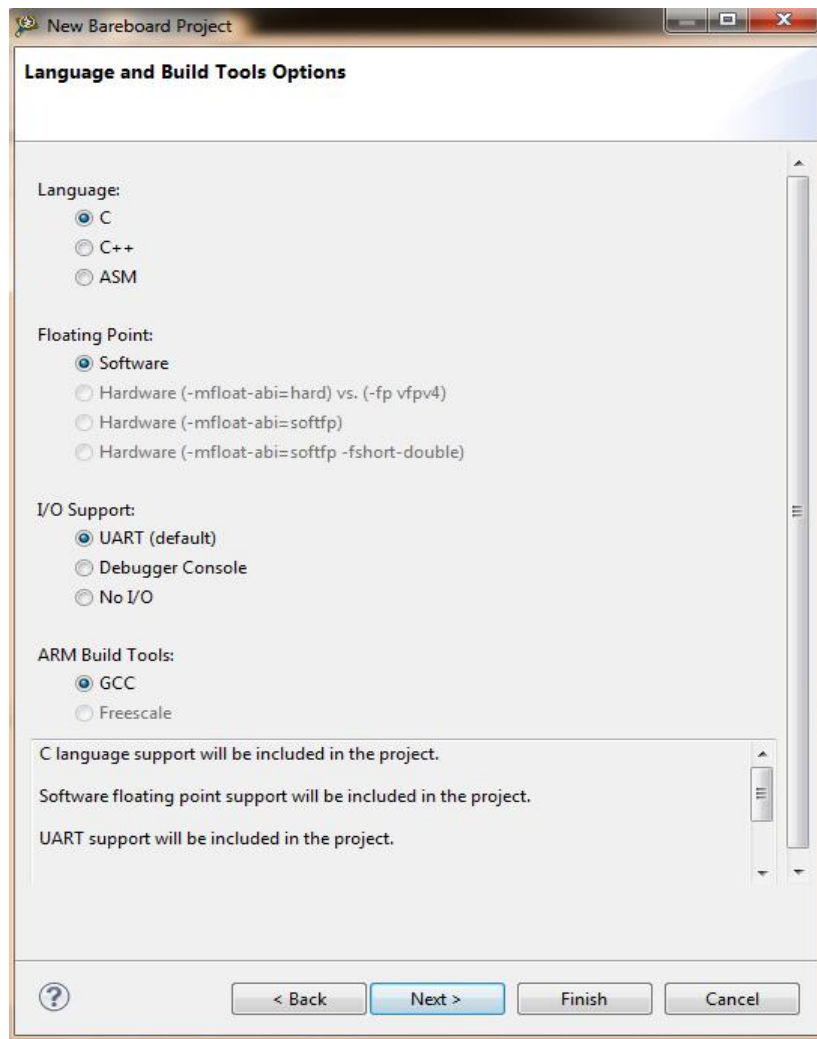


Figura 13. Selectarea limbajului pentru aplicația dezvoltată

6. În fereastra care se va deschide se va selecta dacă se dorește, sau nu, o dezvoltare rapidă a aplicației prin folosirea Processor Expert și dacă se va folosi o perspectivă hardware prin inițializare sau o perspectivă curentă. Pentru aplicația noastră se va folosi o dezvoltare rapidă cu ajutorul Processor Expert și o perspectivă hardware.

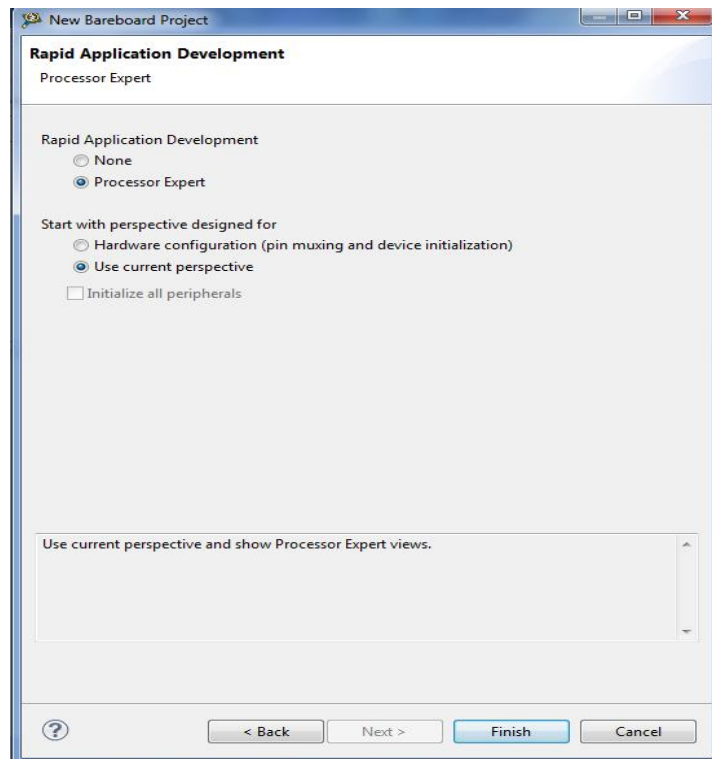


Figura 14. Selectarea modului de dezvoltare al aplicației

7. Următorul pas îl constituie selectarea tipului de procesor utilizat, în cazul nostru se folosește un procesor MKL46Z256VLL4 într-un pachet LQFP de 100 de pini .

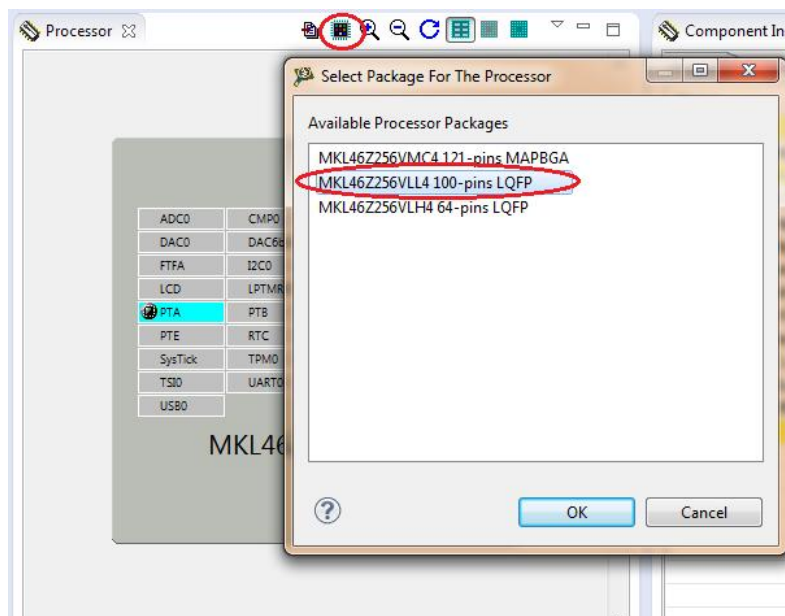


Figura 15. Selectarea procesorului utilizat

În continuare spațiul de lucru va arăta ca în figura de mai jos. Se pot observa fereastra care conține regiștrii procesorului, fereastra în care este reprezentat procesorul, fereastra în care se găsesc componente care pot fi legate la procesor și fereastra în care aceste componente se configurează.

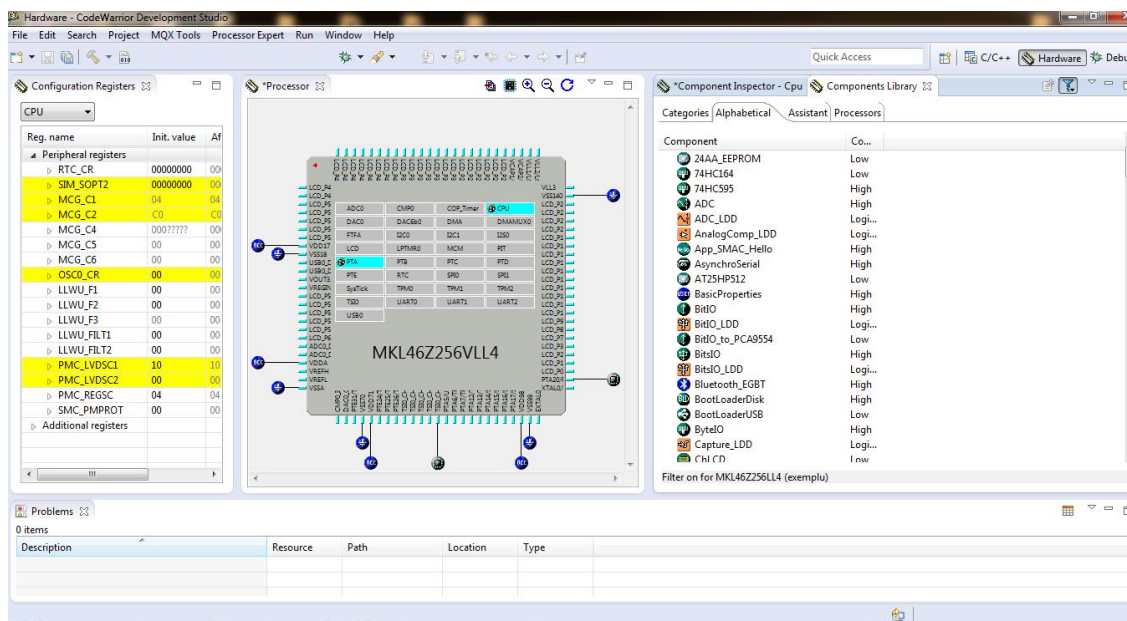


Figura 16. Spațiul de lucru

În cazul în care nu sunt afișate toate ferestrele, pentru adăugarea lor se va selecta, din meniul **Window->Show View** și se adaugă ferestrele necesare.

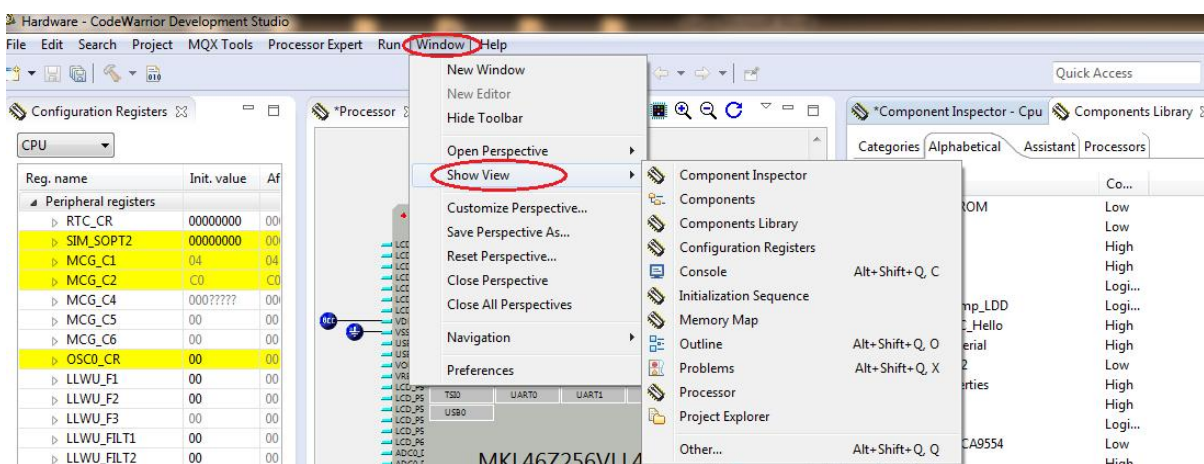


Figura 17. Adăugarea ferestrelor în spațiul de lucru

2.2.4. Crearea unui program în mediul de dezvoltare CodeWarrior v10.5

Aplicația constă în aprinderea celor două LED-uri prezente pe platforma de dezvoltare KL46Z. Pentru aceasta este nevoie de inițializarea acestor componente. Astfel în fereastra **Components Libray** se alege tipul de componentă care se dorește a fi conetată la procesor. LED-urile fac parte din categoria Intrări/Ieșiri Digitale. Apoi se selectează tipul componentei și tipul pinului componentei.

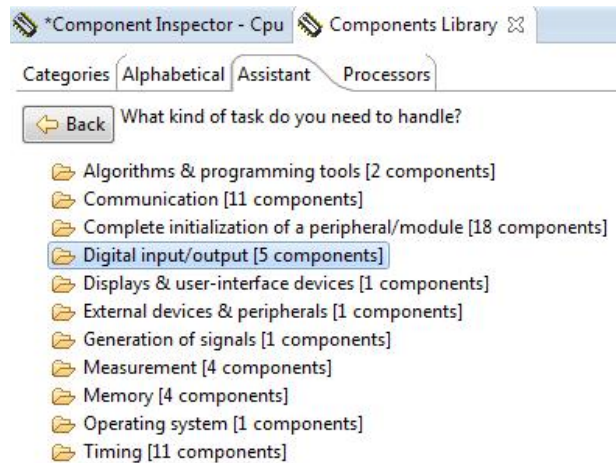


Figura 18. Selectarea componentei care se dorește a fi conectată la procesor

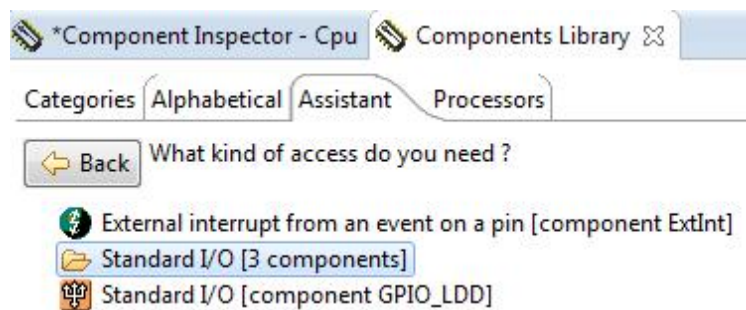


Figura 19. Selectarea tipului componentei

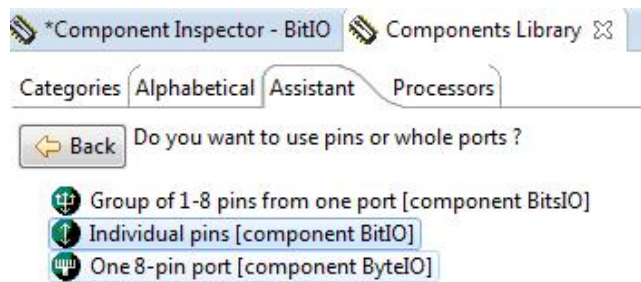


Figura 20. Selectarea tipului pinului componentei care se conectează la procesor

Pentru inițializarea componentei trebuie specificat pinul procesorului la care este conectat LED-ul, în cazul nostru, direcția, dacă este intrare sau ieșire precum și valoarea inițială.

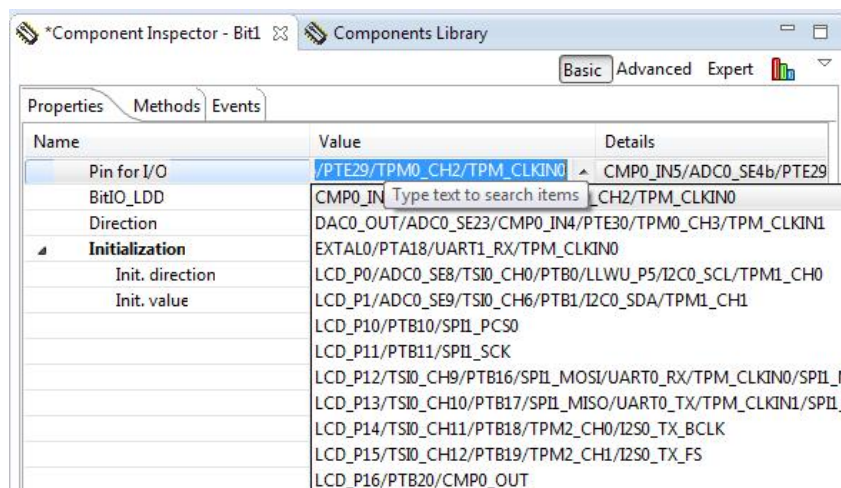


Figura 21. Selectarea pinului la care se conectează LED-ul roșu

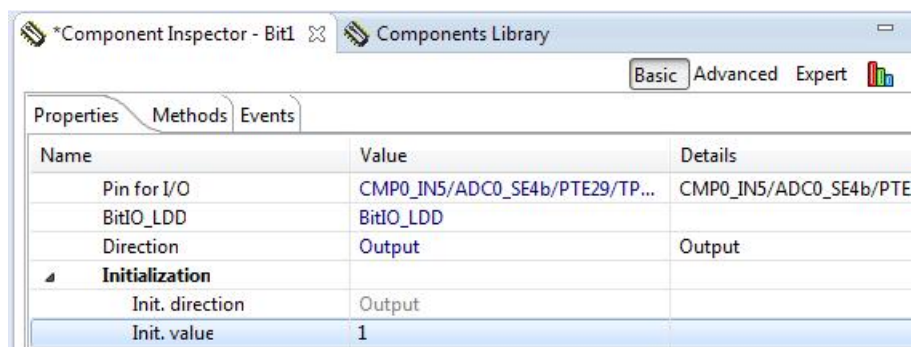


Figura 22. Selectarea direcției și a valorii inițiale

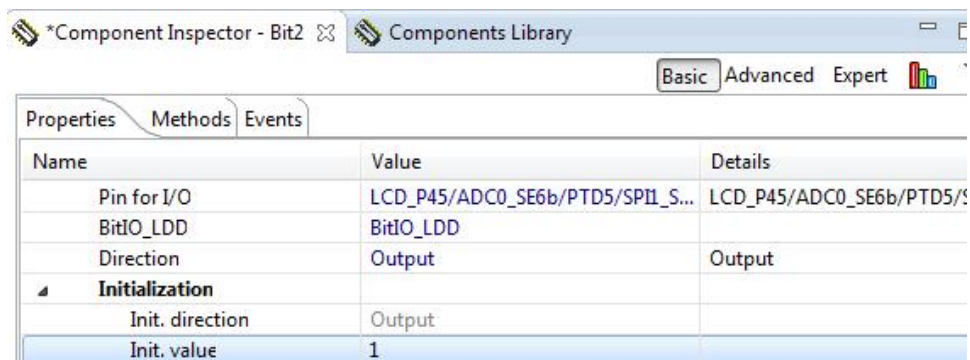


Figura 23. Selectarea pinului la care se conectează LED-ul verde, direcția și valoarea inițială a acestuia

După inserarea tuturor componentelor necesare se selectează butonul C/C++, unde putem scrie programul in limbajul C. In spațiul de lucru se pot observa și componentele adăugate, precum și ferestrele unde se pot inițializa componentele și adăuga altele noi.

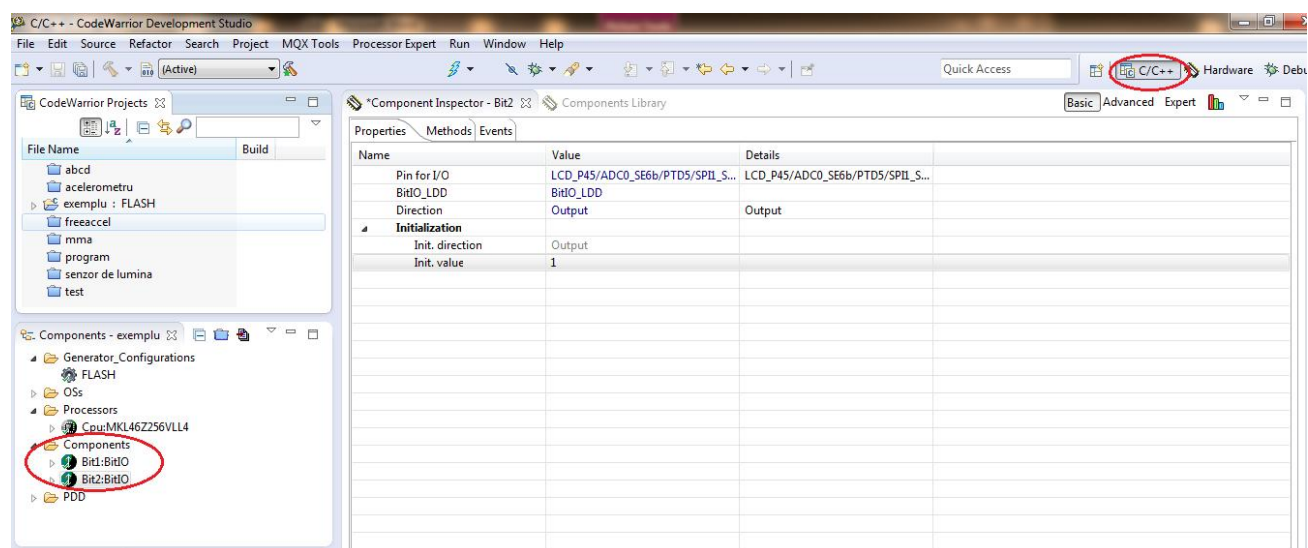


Figura 24. Spațiul de lucru după apăsarea butonului C/C++

Componentele adăugate pot fi redenumite, șterse, salvate ca model sau se pot afla alte proprietăți ale acestuia. De asemenea, se poate genera automat codul pentru componentă.

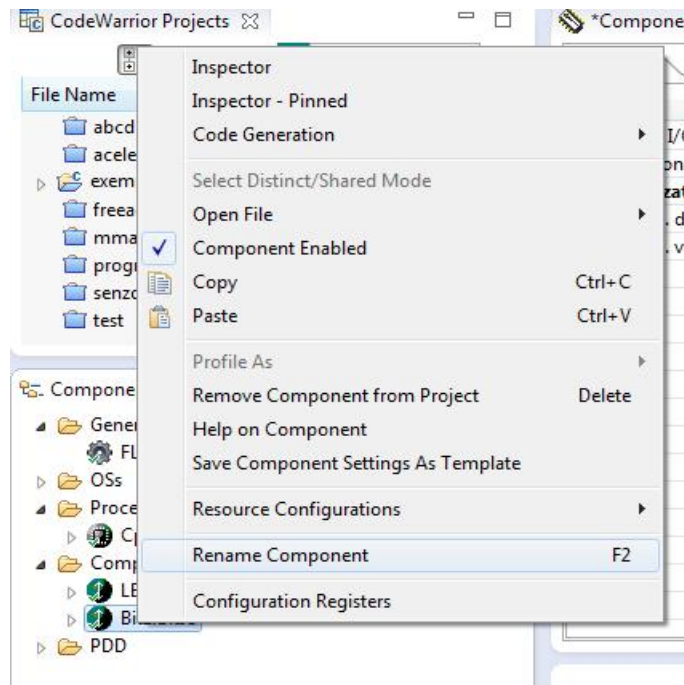


Figura 25. Meniul pentru redenumirea, ștergerea sau salvarea unei component

Funcția `main()` a fost deja creată, ea se află în fișierul **ProcessorExpert.c**, fișier în care vom adăuga și codul nostru. Codul pe care îl vom scrie noi va trebui inserat doar acolo unde scrie comentariul “Write your code here” pentru a nu modifica din codul creat automat de Processor Expert în urma setărilor pe care le-am făcut pentru fiecare component adăugată.

Pentru fiecare componentă în parte, după inițializare, se crează o listă cu instrucțiuni care se pot utiliza pentru aceasta. Astfel pentru aprinderea unui LED vom folosi instrucțiunea **PutVal**, instrucțiune care are rolul de a pune valoarea LED-ului în 0. Pentru ambele LED-uri vom folosi aceeași instrucțiune.

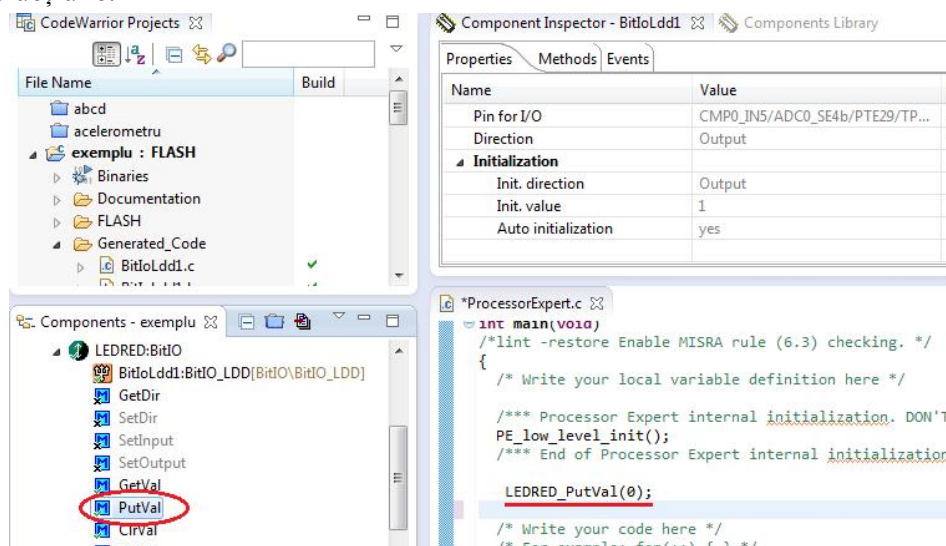


Figura 26. Instrucțiunile componentelor

În cazul în care nu este instalat modulul de ProcessorExpert nu se mai generează automat fișierul ProcessorExpert.c. Acest fișier trebuie să îl creăm noi, trebuie să definim toate porturile pe care le folosim, să scriem adresele la care se găsesc aceste porturi și să facem toate inițializările necesare. Fără ProcessorExpert nu se mai creează cod automat în C pentru componentele pe care le folosim în aplicație, nu mai putem selecta componentele și să facem configurarea lor în ComponentInspector, configurarea lor se va face cu ajutorul regiștrilor specifici portului la care este conectată componenta pe care dorim să o folosim.

Processor Expert Software, Microcontroller Driver Suite, este un sistem de management software care generează cod C pentru a crea, configurarea, optimiza, migra și livra componente software, cum ar fi drivere periferice, pentru procesoare Kinetis și ColdFire +. Acest driver este livrat și instalat ca un produs global, cu Eclipse 4.2 (Juno) IDE. Este, de asemenea disponibil ca un plug-in Eclipse pentru Eclipse 3.7 existent (Indigo) și Eclipse 4.2 (Juno). Driver-ul nu include un compilator sau linker. Îmbinați codul generat într-un sistem de build. Această funcționalitate este integrată în instrumentele de CodeWarrior. Software-ul Processor Expert este compatibil cu toate platformele care utilizează CodeWarrior IDE.

În cazul în care dorim să creăm un nou fișier de tip Sursă (.c) sau de tip Header(.h) se selectează **New** și se alege tipul de fișier dorit.

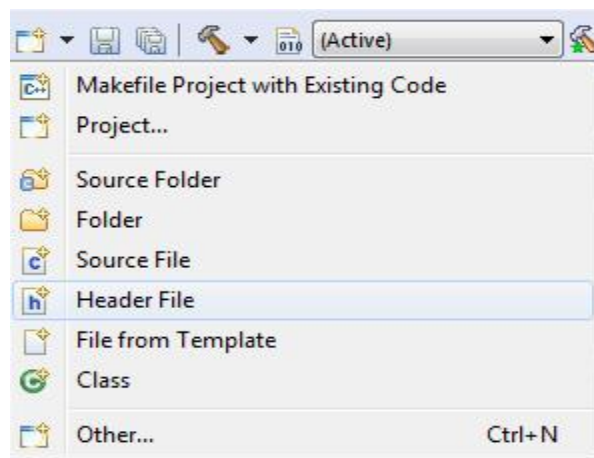


Figura 27. Crearea unui fișier de tip Sursă sau de tip Header

După scrierea tuturor instrucțiunilor necesare pentru programul care se dorește a fi implementat se selectează butonul **Build**, care verifică dacă în programul scris există erori sau nu și locul unde se află erorile.

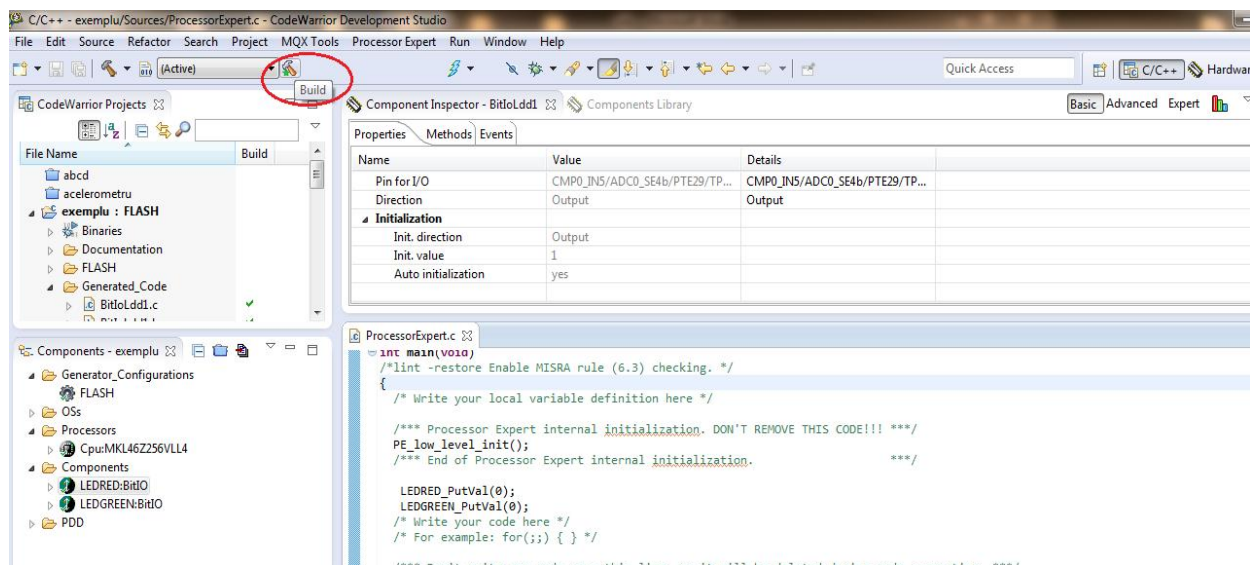


Figura 28. Apăsarea butonului **Build**

Programul nu poate fi implementat pe platformă dacă are erori. Doar după corectarea acestora programul poate fi implementat.

Dacă programul are doar atenționări acesta poate fi depanat.

Pentru implementarea programului pe platforma de dezvoltare KL46Z se apasă butonul de **Debug**.

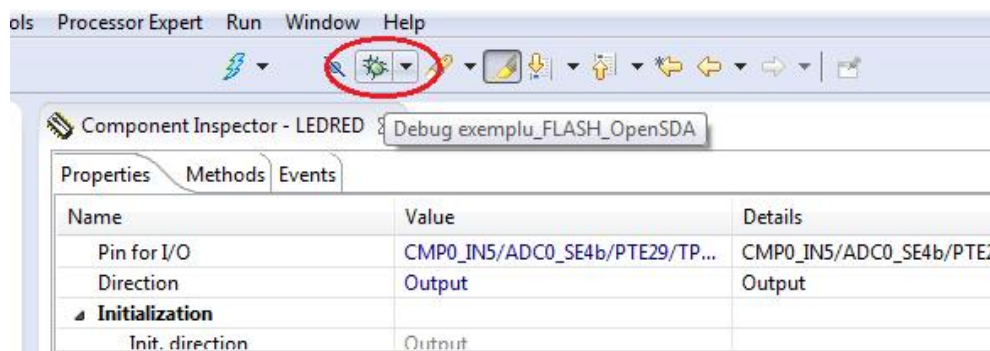


Figura 29. Apăsarea butonului de Debug

După apăsarea butonului de Debug, în spațiul de lucru vor apărea ferestrele care conțin programul în limbajul C, programul transpus în limbaj de asamblare precum și fereastra cu regiștrii procesorului, dacă este nevoie.

Pentru implementarea programului pe platformă se apasă butonul de **Resume**. Dacă programul a fost scris corect, pe platforma de dezvoltare KL46Z se vor vedea LED-urile aprinse, sau după caz rezultatul programului implementat.

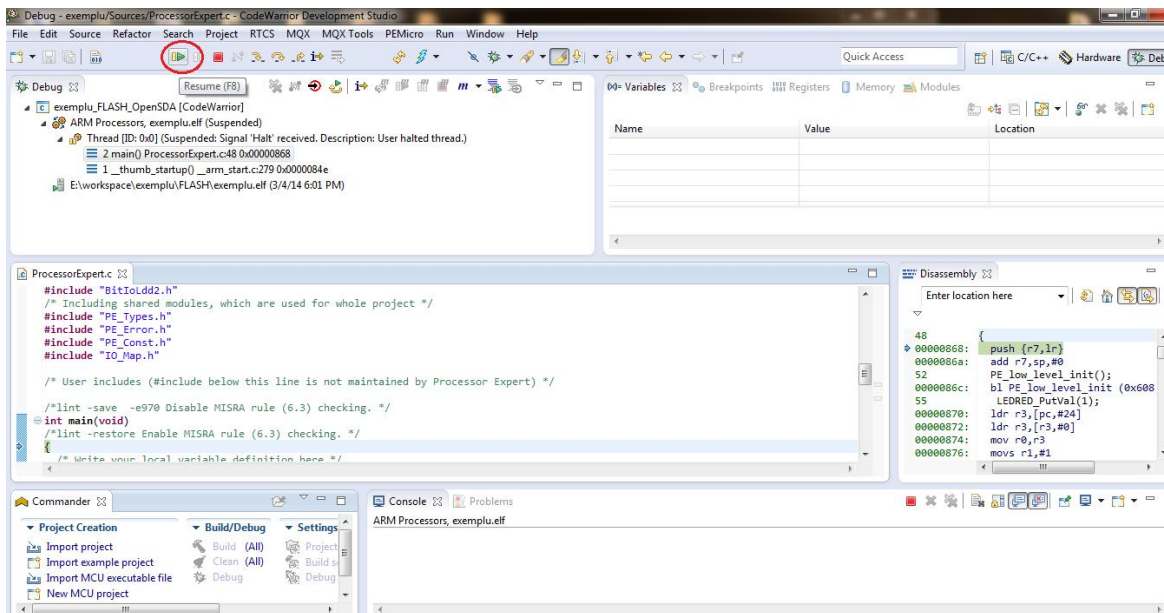


Figura 30. Implementarea programului pe platformă

Dacă avem deja un program scris pentru platforma KL46Z, pentru importarea acestuia, din spațiul de lucru, în fereastra **Commander** se selectează **Import Example Project** sau **Import Project** și în fereastra deschisă se alege calea spre proiectul deja existent.

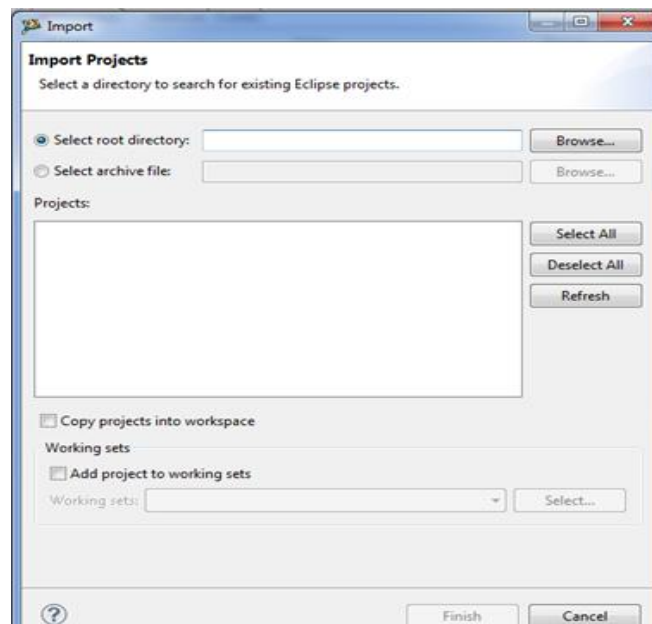


Figura 31. Importarea unui proiect

3. Desfășurarea lucrării

3.1. Probleme rezolvate

3.1.1. Să se creeze un program care realizează aprinderea led-urilor la apăsarea celor două butoane disponibile, de pe platforma de dezvoltare FRDM KL46Z.

Soluție :

Se va porni rezolvarea problemei de la implementarea exemplului de mai sus. În plus se vor adăuga cele două butoane.

Pentru configurarea acestora se va proceda ca la configurarea led-urilor, numai ca vor fi diferiti pini la care sunt conectate aceste butoane. Acești pini se vor regăsi în schematicul platformei de dezvoltare.

Pentru a activa porturile corespunzătoare celor două butoane se selectează “**Alphabetical**” din fereastra “**Component Library**”. Apoi se selectează “**INIT GPIO**”. În “**Component Inspector**” se alege portul corespunzător, în cazul nostru portul C, iar din lista de pini se activează pini corespunzători tastelor, ca în figura de mai jos :

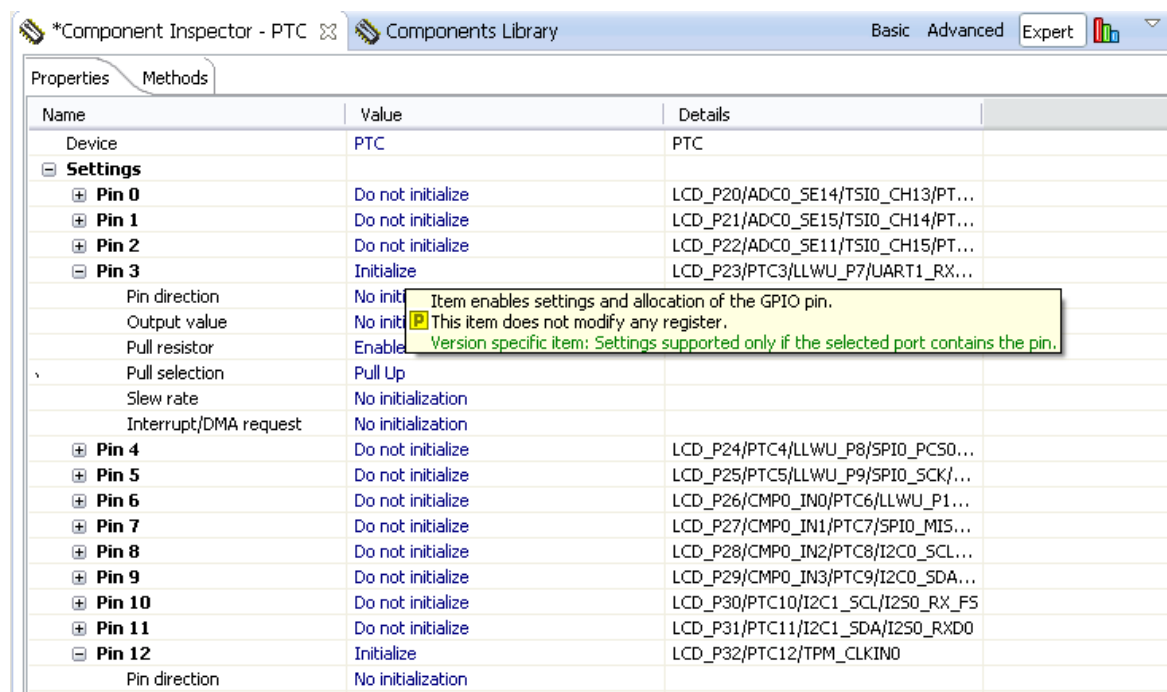


Figura 32. Selectarea portului pentru butoane

Codul programului:

```
**      Filename      : ProcessorExpert.c
**      Project       : ProcessorExpert
**      Processor     : MKL46Z256VLL4
**      Version       : Driver 01.01
**      Compiler      : GNU C Compiler
**      Date/Time     : 2013-10-24, 17:28, # CodeGen: 0
**      Abstract      :
**          Main module.
**          This module contains user's application code.
**      Settings      :
**      Contents      :
**          No public methods
**
**
#####
###*/
/*!
** @file ProcessorExpert.c
** @version 01.01
** @brief
**      Main module.
**      This module contains user's application code.
**/
/*!
**      @addtogroup ProcessorExpert_module ProcessorExpert module
documentation
**      @{
**/
/* MODULE ProcessorExpert */

/* Including needed modules to compile this module/procedure */
#include "Cpu.h"
#include "Events.h"
#include "LEDRED.h"
#include "BitIoLdd1.h"
#include "LEDGREEN.h"
#include "BitIoLdd2.h"
#include "SW1.h"
#include "BitIoLdd3.h"
```

```

#include "SW2.h"
#include "BitIoLdd4.h"
#include "PTC.h"
/* Including shared modules, which are used for whole project */
#include "PE_Types.h"
#include "PE_Error.h"
#include "PE_Const.h"
#include "IO_Map.h"

volatile uint32_t EventCount = 0u;
LDD_TDeviceData *MyTU1Ptr;
LDD_TDeviceData *MyTU2Ptr;
LDD_TError Error;

/* User includes (#include below this line is not maintained by
Processor Expert) */

/*lint -save -e970 Disable MISRA rule (6.3) checking. */
int main(void)
/*lint -restore Enable MISRA rule (6.3) checking. */
{
    /* Write your local variable definition here */

    /*** Processor Expert internal initialization. DON'T REMOVE
THIS CODE!!! ***/
    PE_low_level_init();
    /*** End of Processor Expert internal initialization.
****/

    /* Write your code here */
    /* For example: for(;;) { } */
    int i;
    while(1) {
        LEDRED_PutVal(SW1_GetVal());
        LEDGREEN_PutVal(SW2_GetVal());
    }

    /*** Don't write any code pass this line, or it will be
deleted during code generation. ***/
    /*** RTOS startup code. Macro PEX_RTOS_START is defined by the
RTOS component. DON'T MODIFY THIS CODE!!! ***/
    #ifdef PEX_RTOS_START

```

```

    PEX_RTOS_START();                                /* Startup of the
selected RTOS. Macro is defined by the RTOS component. */
#endif
    /*** End of RTOS startup code.   ***/
    /*** Processor Expert end of main routine. DON'T MODIFY THIS
CODE!!! ***/
    for(;;){}
    /*** Processor Expert end of main routine. DON'T WRITE CODE
BELOW!!! ***/
} /*** End of main routine. DO NOT MODIFY THIS TEXT!!! ***/

/* END ProcessorExpert */
/*!
** @}
**/
/*
**
#####
###
**
**      This file was created by Processor Expert 10.3 [05.08]
**      for the Freescale Kinetis series of microcontrollers.
**
**
#####
###

```

- După compilarea problemei rezolvate, observați ce se întâmplă dacă nu este configurat portul corespunzător.
- Căutați alte metode de setare a pinilor corespunzători tastelor de pe platformă.
- Care metodă vi se pare mai eficientă?

3.1.2. Creați un program care realizează aprinderea celor două led-uri, de pe platforma Freescale Freedom KL46Z. Aprinderea lor se va face astfel:

- La apăsarea primului buton se vor aprinde intermitent, cu o anumită frecvență, la o perioadă de 500ms;
- La apăsarea celui de-al doilea buton se vor aprinde intermitent cu o frecvență diferită de prima frecvență, cu o perioadă de 250 ms;
- Se va folosi un delay de 100 μs pentru a se vizualiza mai bine aprinderea acestora.

La apăsarea butonului de reset circuitul se va reseta.

Soluție :

Pentru rezolvarea problemei se vor parcurge pașii enumerați mai sus, în exemplu. Se vor adăuga cele două led-uri și se vor face setările corespunzătoare. În plus se vor adăuga cele două butoane și se vor selecta porturile I/O pentru aceste butoane. Se vor face configurările necesare. Butonul de reset nu trebuie folosit pentru ca acesta este deja configurat pentru resetarea sistemului.

Se vor adăuga și două timere care vor ajuta la setarea frecvenței pentru fiecare apăsare de buton. Aceste timere vor avea perioadele de 500ms, respectiv 250ms.

Component Inspector - TU1			Component Inspector - TU2		
Properties			Properties		
Name	Value	Details	Name	Value	Details
Module name	LPTMR0	LPTMR0	Module name	TPM0	TPM0
Counter	LPTMR0_CNR	LPTMR0_CNR	Counter	TPM0_CNT	TPM0_CNT
Counter direction	Up		Counter direction	Up	
▲ Input clock source	Internal		▲ Input clock source	Internal	
Counter frequency	512 Hz	512 Hz	Counter frequency	163.84 kHz	163.840 kHz
▲ Counter restart	On-match		▲ Counter restart	On-match	
Period device	LPTMR0_CMR	LPTMR0_CMR	Period device	TPM0_MOD	TPM0_MOD
Period	500 ms	500 ms	Period	250 ms	250 ms
Interrupt	Enabled		Interrupt	Enabled	
Channel list	0		Channel list	0	
▲ Initialization			▲ Initialization		
Auto initialization	no		Auto initialization	no	

Figura 33. Setări pentru selectarea perioadei de aprindere a celor două led-uri

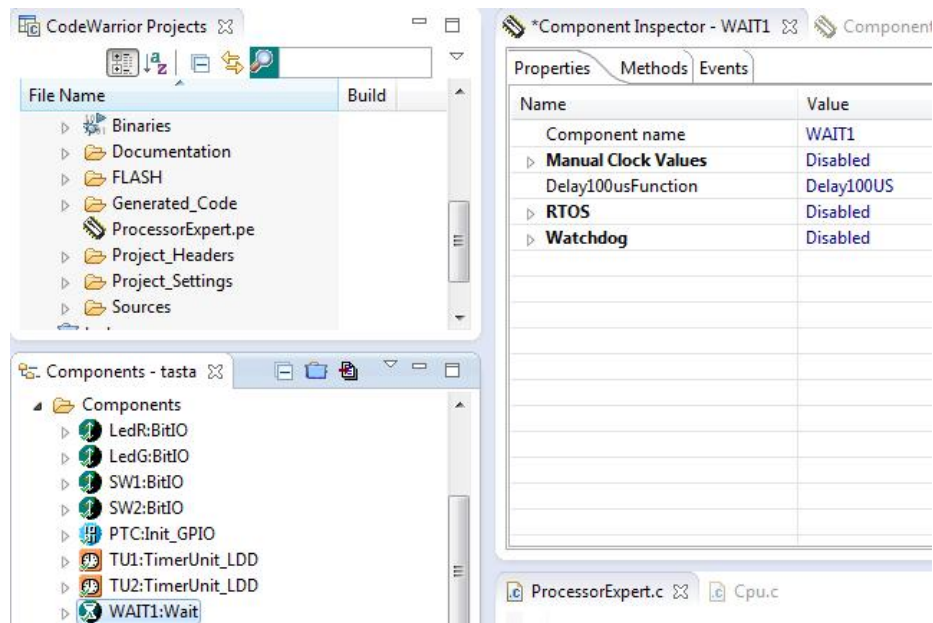


Figura 34. Setările pentru Wait

Pentru a adăuga delay-ul de 100us se va insera din fereastra “**Component Library**”- “**Alphabetical**” component **Wait**.

În codul programului, TU1, respective TU2 reprezintă timer-ul 1, respectiv timer-ul 2. Acestea au funcțiile construite în fișierele TU1.c, TU1.h respectiv TU2.c, TU2.h, iar ele sunt implementate tot în aceste fișiere create automat de Processor Expert.

Variabilele MyTU1Ptr, MyTU2Ptr, Error sunt inițializate în programul principal astfel încât să fie compatibile cu valorile pe care le returnează.

Codul programului:

```
**      Filename      : ProcessorExpert.c
**      Project       : ProcessorExpert
**      Processor     : MKL46Z256VLL4
**      Version       : Driver 01.01
**      Compiler      : GNU C Compiler
**      Date/Time     : 2013-10-24, 17:28, # CodeGen: 0
**      Abstract      :
**          Main module.
**          This module contains user's application code.
**      Settings      :
**      Contents      :
**          No public methods
**
**
#####
###*/
/*!
** @file ProcessorExpert.c
** @version 01.01
** @brief
**          Main module.
**          This module contains user's application code.
**/
/*!
** @addtogroup ProcessorExpert_module ProcessorExpert module
documentation
** @{
**/
/* MODULE ProcessorExpert */
```

```

/* Including needed modules to compile this module/procedure */
#include "Cpu.h"
#include "Events.h"
#include "LedR.h"
#include "BitIoLdd1.h"
#include "LedG.h"
#include "BitIoLdd2.h"
#include "SW1.h"
#include "BitIoLdd3.h"
#include "SW2.h"
#include "BitIoLdd4.h"
#include "PTC.h"
#include "TU1.h"
#include "TU2.h"
#include "WAIT1.h"
/* Including shared modules, which are used for whole project */
#include "PE_Types.h"
#include "PE_Error.h"
#include "PE_Const.h"
#include "IO_Map.h"

volatile uint32_t EventCount = 0u;
LDD_TDeviceData *MyTU1Ptr;
LDD_TDeviceData *MyTU2Ptr;
LDD_TError Error;

/* User includes (#include below this line is not maintained by
Processor Expert) */

/*lint -save -e970 Disable MISRA rule (6.3) checking. */
int main(void)
/*lint -restore Enable MISRA rule (6.3) checking. */
{
    /* Write your local variable definition here */

    /*** Processor Expert internal initialization. DON'T REMOVE
THIS CODE!!! ***/
    PE_low_level_init();
    /*** End of Processor Expert internal initialization.
****/

```

```

    /* Write your code here */
    /* For example: for(;;) { } */

// int i;

while(1){
    if(SW2_GetVal()==0){

        MyTU1Ptr = TU1_Init((LDD_TUserData *)NULL);          /*
Initialize the device */
        Error = TU1_Enable(MyTU1Ptr);
        Error = TU2_Disable(MyTU2Ptr);
    }

    if(SW1_GetVal()==0){
        MyTU2Ptr = TU2_Init((LDD_TUserData *)NULL);
        Error = TU2_Enable(MyTU2Ptr);
        Error = TU1_Disable(MyTU1Ptr);
    }

    LedG_PutVal(EventCount);
    LedR_PutVal(EventCount);
}

/** Don't write any code pass this line, or it will be
deleted during code generation. ***/
/** RTOS startup code. Macro PEX_RTOS_START is defined by the
RTOS component. DON'T MODIFY THIS CODE!!! ***/
#ifdef PEX_RTOS_START
    PEX_RTOS_START();          /* Startup of the
selected RTOS. Macro is defined by the RTOS component. */
#endif
/** End of RTOS startup code. ***/
/** Processor Expert end of main routine. DON'T MODIFY THIS
CODE!!! ***/
for(;;){}
/** Processor Expert end of main routine. DON'T WRITE CODE
BELOW!!! ***/
} /** End of main routine. DO NOT MODIFY THIS TEXT!!! ***/

```

```

/* END ProcessorExpert */
/*!
** @}
*/
/*
**
#####
###
**
**      This file was created by Processor Expert 10.3 [05.08]
**      for the Freescale Kinetis series of microcontrollers.
**
**
#####
###

```

- După implementarea codului propus, căutați alte metode de creare a acestui program
- Modificați perioadele timere-lor și explicați ce se întâmplă.
- Modificați perioada de întârziere și explicați ce se întâmplă.

3.2. Probleme propuse

După parcurgerea breviarului teoretic se vor implementa problemele rezolvate observând modul de funcționare direct folosind platforma de dezvoltare FRDM KL46Z, iar dacă este cazul se vor căuta alte metode de rezolvare a acestora. După implementarea problemelor rezolvate se va observa modul de implementare a algoritmilor, se vor adăuga noi componente conform breviarului teoretic și se vor cauta algoritmi care să evidențieze funcționarea componentelor adăugate.

Se vor căuta soluții și se vor implementa următoarele probleme propuse:

- 3.2.1. Să se realizeze un program care să facă ca cele două led-uri să se aprindă intermitent, cu frecvențe diferite.
- 3.2.2. Să se creeze un program prin care led-urile se vor aprinde la apăsarea unui buton, iar prin apăsarea celui alt să se stingă.
- 3.2.3. Să se realizeze un program prin care la apăsarea unui buton led-urile se vor aprinde. La apăsarea celui de-al doilea buton acestea vor licări intermitent, iar la a doua apăsare a celui de-al doilea buton acestea să se stingă.

3.3. Întrebări

- 3.3.1. Ce periferice se găsesc pe platforma de dezvoltare de la Freescale FRDM KL46Z ?
- 3.3.2. Ce fel de procesor se găsește pe această platformă și principalele caracteristici ale acestuia.
- 3.3.3. Ce rol are driver-ul Processor Expert?
- 3.3.4. Ce limbaje de programare pot fi folosite pentru programarea platformei de dezvoltare FRDM KL46Z?